University
Mohammed VI
Polytechnic

College of Computing

Deliverable #: Lab-7/ Physical Design, Security and Transaction Management,Transactions and Concurrency Control

Data Management Course

UM6P College of Computing

Professor: Karima Echihabi **Program:** Computer Engineering

Session: Fall 2025

Team Information

Team Name	StarsDB
Member 1	Nouha Talssi
Member 2	Mariam Said
Member 3	Tilamsane Wissal
Member 4	Alae Sabir
Member 5	Amina Sidki
Repository Link	`https://github.com/StarsDB`

1 -Introduction

This deliverable focuses on the physical design and transaction management aspects of the MNHS database.The main objective of the lab is to understand and demonstrate how design choices like indexes,partitioning,tablespaces,and storage layout can impact the performance and efficiency of database operations and queries especially while dealing with voluminous data.Additionally,we will explore ACID properties,schedules,conflict serializability,2PL protocol,and deadlock handling to ensure data integrity and consistency during concurrent transactions especially in a multi-user environment.
The deliverable is divided into Two main parts : Physical Design, Security and Transaction Management plus Transactions and Concurrency Control.

2 -Methodology

To assure a structured approach to this lab, we followed these steps:

2.1 -Part1: Physical Design, Security and Transaction Management

- We began by analyzing the views and queries from lab6 to identify areas that could be optimized through indexing and partitioning.
- We populated the MNHS database with a large volume of data to simulate real-world scenarios and test the performance of our physical design choices.
- We implemented various indexes and partitioning strategies based on the query patterns identified earlier.
- We then evaluated the performance of the database before and after indexing using MySQL command line and a query selected by choice to measure execution time and resource usage.
- We then visualized the impact of indexing on query performance using appropriate graph.

2.2 -Part2: Transactions and Concurrency Control

- We started by reviewing each scenario provided in the lab to determine the ACID properties that are being satisfied or violated.
- We than implemented atomic transactions in MySQL ensuring that each transaction either completes fully or not at all.
- We first analyzed given schedules to identify their types .Than we determined conflict serializability of another given schedule.
- We implemented the 2PL protocol in MySQL to manage concurrent transactions and ensure serializability.

- We simulated deadlock scenarios and implemented deadlock detection and resolution strategies to handle them effectively.

3 Implementation & Results

3.1 -Part1: Physical Design, Security and Transaction Management

3.1.1 Index Design

- **UpcomingByHospital View:**

```
CREATE INDEX idx_Appointment_Status_CAID ON Appointment
    (Status,CAID);
--Table :Appointment
--Attribute list :status, CAID
--Index Type : B+Tree
```

Listing 1: Index for UpcomingByHospital View

  - This index improves the filtering condition in the A.Status = 'Scheduled'.Since the status is the first column in the composite index, the databse can directly locate only the rows where the status is Scheduled, instead of scanning the entire Appointment table.
  - The index also improves the join condition A.Ciad = C.CAID After filtering by Status, the remaining CAID values are already indexed and sorted, allowing the database to efficiently match them with the ClinicalActivity table during the join.
  - The composite index is well chosen because it follows the query structure first an equality filter on Status then a joing using CAID this ordering allows the databse to fully benefit from the B+ tree index structure .
  - This index significantly speeds up read operations which is ideal for the view but slows down write operations insert , delete , update since the index must be updated whenever the Appointment table changes.

- **StaffWorloadThirty View:**

```
CREATE INDEX  CA_idx_date ON ClinicalActivity(Date,CAID)
--Table :ClinicalActivity
--Attribute list :Date
--Index Type : B+Tree
```

Listing 2: Index for StaffWorloadThirty View

  - This index is designed to optimize queries that filter clinical activities based on date.
  - Given the high number of different dates in the ClinicalActivity table, this index will significantly enhance query performance by allowing faster filtering.

- The index does introduce some overhead during data modifications and insertion, but the improved performance justifies this cost.
- We thought about creating another index for the appointment status attribute, but since it has low cardinality (only a few distinct values), it would not provide significant performance benefits compared to the overhead it would introduce.

- **PatientNextVisit View:**

```
CREATE INDEX  idx_clinicalactivity_date ON
   ClinicalActivity(Date,Time)
--Table :ClinicalActivity
--Attribute list :Date,Time
--Index Type : B+Tree
```

Listing 3: Index for UpcomingByHospital View

- This index is designed to optimize queries that select upcoming clinical activities based on date and time.
- Since there are a lot of different dates and times for clinical activities,then using an index on these attributes will significantly speed up the retrieval process.
- The index does introduce additional maintenance cost during insertions and updates, but the performance gains for read operations outweigh these costs.So overall, the overhead is justified.

- **AppQuery:**

```
--For this Query we will create different indexes:
CREATE INDEX idx_hid ON Department(HID);
--Table: Department
--Attribute list: HID
--Index Type: B+Tree
CREATE INDEX idx_dep_id_date ON ClinicalActivity(DEP_ID,
   Date);
--Table: ClinicalActivity
--Attribute list: DEP_ID, Date
--Index Type: B+Tree
```

Listing 4: Index for AppQuery

- The first index ON Department(HID) reduces the cost of the join between the Hospital and Department tables by directly locating the departments belonging to each hospital avoiding unnecessary scans of the whole Departments table.(The overhead of maintaining this index is justified given the expected performance improvements for this query.
- The second composite index ON ClinicalActivity(DEP_ID, Date) could be used by the optimizer to reduce the cost of the join between ClinicalActivity and Departments table then efficiently filter rows by date by reducing the number of processed tuples. (The overhead of maintaining this index is justified given the expected performance improvements for this query.)

3.1.2 Partitioning:

- **ClinicalActivity Table:**For this table,we saw that the best partitioning strategy would be RANGE partitioning on Date attribute depending on the year or the month of the clinical activity.
  -If we take for example a query that retrieves all clinical activities scheduled for the next month,without partitioning , the database would have to scan the entire ClinicalActivity tables to find the relevant rows.This can be very time-consuming and costly in terms of performance especially with large data.However, with RANGE partitioning the database could scan only the partitions that correspond to the next month.This only would significantly reduce the amount of data that needs to be processed,leading to faster query execution times .
  -About the access for old data , RANGE partitioning also helps in managing and archiving old data efficiently.Since each partition corresponds to a specific range of dates,old partitions can be easily identified,deleted or archived without doing each modification on each row individually .
  -Also,if we consider different queries that filter on columns other than Date, the database will still scan the necessary partitions or all partitions depending on the filter condition.What we could say is :
  - No data will be lost or blocked because of this partitionning strategy.
  - The performance will be improved for queries filtering on Date attribute.However,for queries filtering on other attributes without Date filter,the performance might not see significant improvements since the database may need to scan multiple or all partitions.
  - Overall,the drawback is minor compared to the benefits gained for queries filtering on Date attribute.

- **Stock Table:**For this table we saw that the best partitioning would be per hospital or group of hospitals.
  -This partitioning strategy would be beneficial for queries that filter by hospital for example:

```
1 SELECT * FROM Stock WHERE HID = 1001 ;
```

  In this case,the database would only need to access the partition corresponding to HID 1001 rather than scanning the entire Stock table.This would significantly reduce the amount of data that needs to be processed,leading to faster query execution times.
  Also,when we use aggregation functions like SUM or AVG grouped by HID,the database can process each partition separately and then combine the results.
  -It's true that this partitioning is beneficial in many cases.However, if some hospitals have significantly more stock than others,this could lead to uneven data distribution across partitions,resulting in potential performance issues for queries that need to access multiple partitions.
  -Also,while using JOIN operations betweenStock and other tables ,the database may need to access multiple partitions if the join condition involves HID from different hospitals.This could lead to increased complexity and potential performance overhead.

-Overall,the benefits of partitioning by HID generally outweigh the drawbacks,especially for queries that frequently filter or aggregate data by hospital.

3.1.3 Tablespaces and Storage Layout:

We started by populating our MNHS database with a large volume of data to simulate real-world scenarios and test the performance of our physical design choices.
We then executed one SQL Query three times in a row :

```sql
SELECT
    D.Name AS DepartmentName,
    COUNT(*) AS NbAppointments
FROM ClinicalActivity_copy C
JOIN Appointment A ON A.CAID = C.CAID
JOIN Department D ON C.DEP_ID = D.DEP_ID
WHERE A.Status = 'Scheduled'
  AND C.Date >= '2024-01-01'
  AND C.Date < '2025-01-01'
GROUP BY D.Name;
```

After that,we measured the execution time and used the EXPLAIN command to analyze the execution plan before and after implementing the idx_clinicalactivity_date index on ClinicalActivity(Date,Time) table.

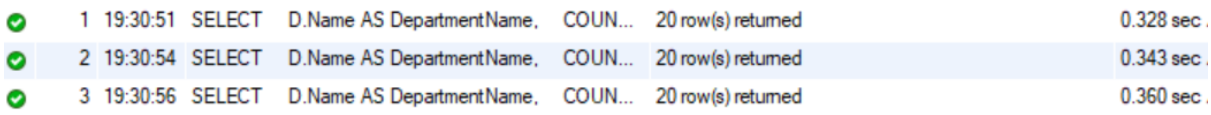


Figure 1: Execution time of the query without index

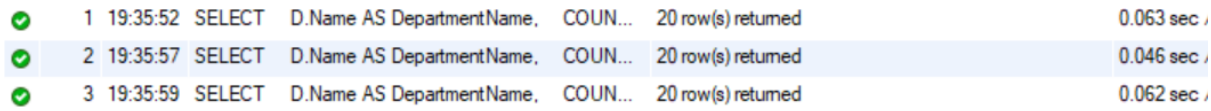
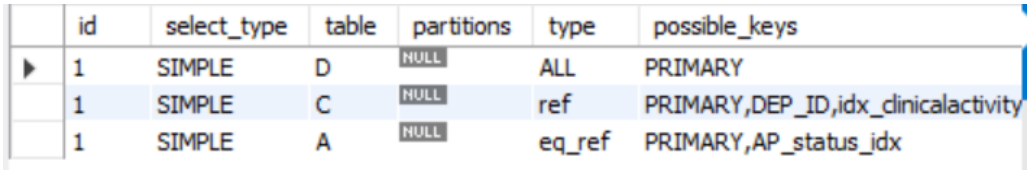


Figure 2: Execution time of the query with index



id	select_type	table	partitions	type	possible_keys
1	SIMPLE	D	NULL	ALL	PRIMARY
1	SIMPLE	C	NULL	ref	PRIMARY,DEP_ID,idx_clinicalactivity
1	SIMPLE	A	NULL	eq_ref	PRIMARY,AP_status_idx

Figure 3: Execution plan of the query without index

From the results above,we can see that the execution time of the query significantly decreased after adding the index.Before,the average time was 0,344 second to scan 100.000 rows.After adding the index,the average time dropped to 0,057 second which represents 84 percent of the before index time.

	key	key_len	ref	rows	filtered	Extra
▶	NULL	NULL	NULL	20	100.00	Using temporar
ity_date	DEP_ID	4	mnhs1.D.DEP_ID	1	39.00	Using where
	PRIMARY	4	mnhs1.C.CAID	1	34.16	Using where

Figure 4: Execution plan of the query without index

	id	select_type	table	partitions	type	possible_keys
▶	1	SIMPLE	A	NULL	ref	PRIMARY,AP_status_idx
	1	SIMPLE	C	NULL	eq_ref	PRIMARY,DEP_ID,idx_clinicalactivity
	1	SIMPLE	D	NULL	eq_ref	PRIMARY

Figure 5: Execution plan of the query with index

The execution plan also shows that the database utilized the index to optimize the query execution.After adding the index, the access type changed from full scan to index scan , indicating that the database was able to leverage the index to quickly locate the relevant rows based on the Date filter condition.It also helped enable faster and better join operations and allow early filtering of rows.

These results demonstrate the effectiveness of indexing in improving query performance,especially for large datasets where scanning the entire table would be inefficient.

3.1.4 Visualizing the Impact of Indexing:

- **subject query:**

```sql
SELECT
    H.Name AS HospitalName,
    C.Date AS ApptDate,
    COUNT(*) AS ScheduledCount
FROM Appointment A
JOIN ClinicalActivity C ON A.CAID = C.CAID
JOIN Department D       ON C.DEPID = D.DEPID
JOIN Hospital H         ON D.HID  = H.HID
WHERE A.Status = 'Scheduled'
  AND C.Date >= CURDATE()
  AND C.Date < DATE_ADD(CURDATE(), INTERVAL 14 DAY)
GROUP BY
    H.Name,
    C.Date;
```

- **Index used:**

```sql
CREATE INDEX idx_ClinicalActivity
ON ClinicalActivity ( Date,CAID, DEPID ) ;
```

- **Visual comparison of the impact of indexing:**

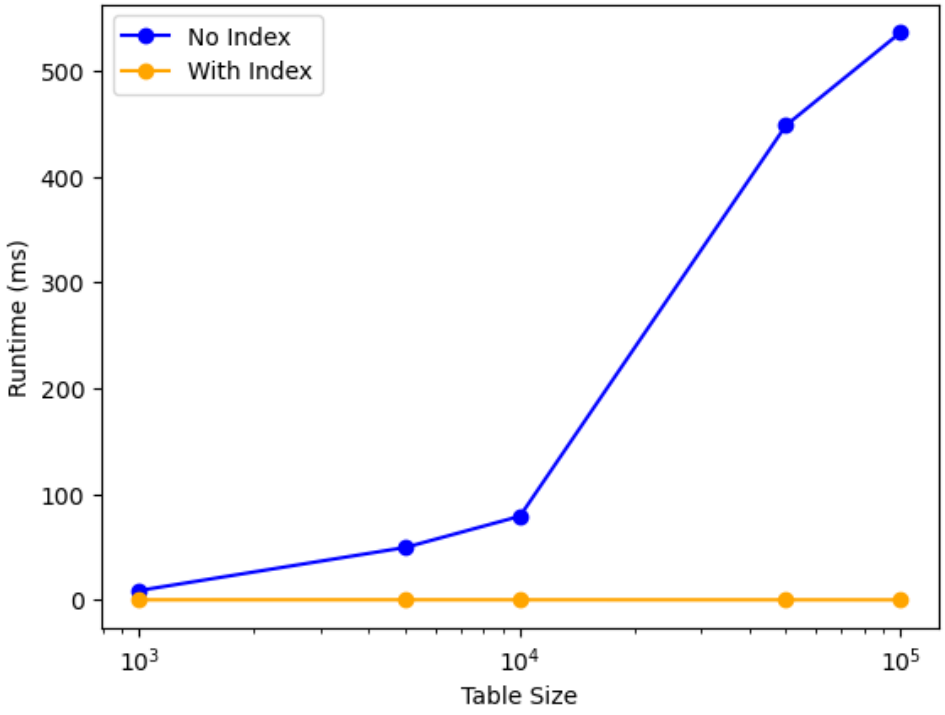


Figure 6: Query Runtime by Table Size

Conclusion: As the table size grows, the runtime without indexes increases sharply, while the indexed runtime stays almost flat, causing the gap between the two curves to widen significantly. This shows that indexing becomes increasingly beneficial for large datasets

3.2 Part2: Transactions and Concurrency Control

3.2.1 Revisiting ACID Transactions

For each scenario provided in the lab,we analyzed the ACID properties that are being satisfied or violated as follows:

- **Exemple 1:**
  - **Atomicity:Violated(temporarily).**Because the insertion of the expenses and the update of the claim are not in the same.One was before the crash and the other after the crash until the retry happened.
  - **Consistency:Satisfied.**The database remains in a consistent state before and after the crash.
  - **Isolation:Satisfied.**Since there are no concurrent transactions mentioned in the scenario.(We can either assume it's satisfied or not applicable in this case.)
  - **Durability:Satisfied.**Once the transaction is retried and completed successfully,the changes are permanent even in the event of a crash.
- **Example 2:**

- **Atomicity:Satisfied.**Because both transactions completed successfully(both receptionists received the confirmation.)
- **Consistency:Satisfied(Assumed).**Their were no violation details in this scenario so we assume the database remains in a consistent state before and after the transactions.
- **Isolation:Violated.**In this case,both receptionists were able to book the last appointment slot concurrently,resulting in double booking.This creates an inconsistent state because only one slot actually exists.
- **Durability:Satisfied(Assumed).**Once the transactions are completed successfully,the changes are permanent even in the event of a crash.(no violation details)

- **Example 3:**
  - **Atomicity:Satisfied.**Since the updating will not appear in the application until the save button is clicked,if the user decides to cancel the changes before saving,the entire transaction is aborted and no changes are made to the database.
  - **Consistency:Satisfied(Assumed).**There were no violation details in this scenario so we assume the database remains in a consistent state .
  - **Isolation:Satisfied(Assumed).**Even if we assumed that isolation is satisfied it is not guaranteed,since Staff B cannot see the changes until Staff A commits but he doesn't actually commit changes so we connot talk about isolation here.
  - **Durability:Satisfied(Assumed).**Once Staff A clicks the save button and the transaction is completed successfully,the changes are permanent even in the event of a crash.(it isn't explicitly mentioned but we assume so.)

- **Example 4:**
  - **Atomicity:Satisfied.**Because all of the transaction failed after the power outage and no data was saved.
  - **Consistency:Satisfied(Assumed).**The database remains in a consistent state .(no violation details)
  - **Isolation:Satisfied(Assumed).**Since there are no concurrent transactions mentioned in the scenario.(We can either assume it's satisfied or not applicable in this case.)
  - **Durability:Violated.**Because the power outage caused the loss of all unsaved data entered during the session before the outage.Which violates the principle of durability.

- **Example 5:**
  - **Atomicity: Satisfied** Only one operation is executed as a single transaction.
  - **Consistency: Satisfied.** (" The system never records negative stock or incorrect totals") → data is still valid after the transaction

- **Isolation: Satisfied.** ("regardless of how many pharmacists are updating stock concurrently")→ even when two transactions are committed at the same time they do not affect the end result.
- **Durability: Satisfied.** ("The pharmacy module ensures that every time a medication is dispensed, the corresponding `Stock.Qty` is reduced by exactly the dispensed amount") → No data is lost during / after the transaction

3.2.2 Implementing Atomic Transactions in MySQL

- Atomic scheduling of an appointment:
  - **a)**

```
START TRANSACTION
INSERT INTO ClinicalActivity (CAID,IID,STAFF_ID,DEP_ID,
   Date,Time)
VALUES(001,002,003,004,'2025-12-14','10:00:00');

INSERT INTO Appointement (CAID,Reason,Status)
VALUES(001,'General check-up','Scheduled');

IF @@ERROR_COUNT > 0 THEN
    ROLLBACK;
    SELECT ' Transaction rolled back due to error .';
ELSE
    COMMIT
    SELECT ' Appomitement scheduled successfully. ';
END IF ;
```

Listing 5: Atomic scheduling of an appointment

  - **b)** - The atomicity property is enforced by wrapping both INSERT in START TRANSACTION ... COMMIT/ROLLBACK , so we ensure if the first insert fails nothing is committed , if the second insert fails , the first is rolled back . - In auto-commit , each insert is its transaction , if the first insert succeeds and the second fails , you'd have a ClinicalActivity without a corresponding appointment (this could violates the business logic and leaves the database in an inconsistent state).

- Atomic update of stock and expense :
  - **a)**

```
This is the pseudocode
BEGIN TRANSACTION
TRY
Updates Stock.Qty for all medications dispensed in a
   given    Prescription (using Includes),
Relies on your trigger logic to recompute the
   corresponding Expense.Total for the underlying
   ClinicalActivity
COMMIT TRANSACTION
```

```
CATCH ERROR
  ROLLBACK TRANSACTION
END TRY
```

Listing 6: Atomic update of stock and expense

– **b)** -The most important ACID property here is isolation, ensuring that concurrent transactions do not interfere with each other and lead to inconsistent stock levels or expense totals.Because if there was little medication stock and two patients try to get the same medications at the same time without proper isolation , it will occur in an error or inconsistent state.

3.2.3 Identifying Types of Schedules:

1. The S1 schedule and S2 schedules are equivalent.Because they both result in the same final state of the database after executing all transactions,regardless of the order of operations.They both have the same read and write operations on the same data items following the same order of transactions.

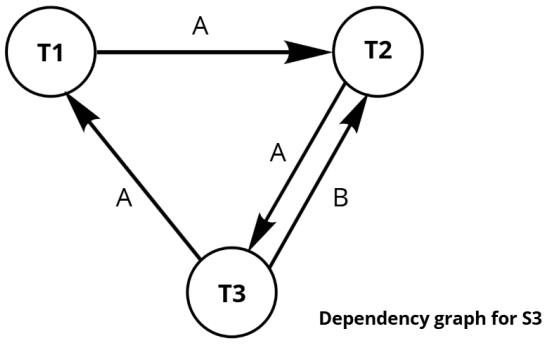
2. The S1 schedule is serializable because it can be transformed into a serial schedule without changing the outcome of the transactions.The operations of T1 and T2 can be reordered to execute all operations of T1 followed by all operations of T2 or vice versa without affecting the final state of the database.One equivalent serial schedule is T1 followed by T2(S2).Also, T2 followed by T1 is another equivalent serial schedule.

3.2.4 Conflict Serializability

Schedule S_3:

$$S_3 = R_1(A), W_2(A), R_3(A), W_1(A), W_3(B), R_2(B)$$

1) dependency graph for S_3



2) Is S_3 Conflict Serializable?

The dependency graph is cyclic. More precisely, it contains two cycles:

- **Cycle 1:**

$$T_1 \to T_2 \to T_3 \to T_1$$

- $R_1(A)$, $W_2(A) \Rightarrow T_1 \rightarrow T_2$
- $W_2(A)$, $R_3(A) \Rightarrow T_2 \rightarrow T_3$
- $R_3(A)$, $W_1(A) \Rightarrow T_3 \rightarrow T_1$

- **Cycle 2:**

$$T_2 \rightarrow T_3 \rightarrow T_2$$

- $W_2(A)$, $R_3(A) \Rightarrow T_2 \rightarrow T_3$
- $W_3(B)$, $R_2(B) \Rightarrow T_3 \rightarrow T_2$

Conclusion: According to the conflict serializability theorem, since the dependency graph contains cycles, the schedule S_3 is **not conflict serializable**.

3.2.5 2PL protocol:

- **Schedule 1:** This schedule follows the 2PL protocol. Because this schedule is the application of T_1 followed by T_2, and in each transaction all locks are acquired before any are released (growing phase followed by shrinking phase).
- **Schedule 2:** This schedule does not follow the 2PL protocol. Because before the last read operation $R_1(B)$ of the schedule, we have an exclusive lock held by $W_2(B)$ from T_2 which is not released yet. So T_1 is trying to acquire a shared lock on B while T_2 still holds an exclusive lock on B, which violates the 2PL protocol.
- **Schedule 3:** This schedule does follow the 2PL protocol. Because like Schedule 1, it is a succession of three transactions T_1, T_2 and T_3 where each transaction acquires all necessary locks before releasing any locks and the transactions are for different data items.
- **Schedule 4:** This schedule does not follow the 2PL protocol. Because of two reasons:
  1. We have a read operation $R_2(A)$ in T_2 that occurs after the write operation $W_1(A)$ in T_1 without T_1 releasing its exclusive lock on A first. (The growing phase of T_1 is not complete before the shrinking phase of T_2 starts).
  2. We have a write operation $W_2(B)$ in T_2 that occurs after the read operation $R_1(B)$ in T_1 without T_1 releasing its shared lock on B first. (The growing phase of T_1 is not complete before the shrinking phase of T_2 starts).

3.2.6 Deadlocks in MNHS

1. **Wait-for graph:**
   It contains two transactions T_1 and T_2 with the following edges:
   - An edge from T_1 to T_2 ($T_1 \rightarrow T_2$)
   - An edge from T_2 to T_1 ($T_2 \rightarrow T_1$)
2. - Since there is a deadlock, we could see that T_1 is waiting for T_2 to release the lock on B while T_2 is waiting for T_1 to release the lock on A.

- The cycle appears here: $T_1 \rightarrow T_2 \rightarrow T_1$
- The DBMS periodically checks the wait-for graph to detect cycles. Once a cycle is found, the DBMS chooses one transaction as a victim and aborts it. The aborted transaction releases all its locks, allowing the other transaction to finish.

4 Discussion

In this lab , we didn't face much technical challenges.The hard part of the lab was in accurately understanding each concept to successfully answer the questions provided in the lab instructions.Some of these challenges were:

- Understanding of indexing and partitioning concept and its use in optimizing time consumption in query execution.
- Balancing the trade-offs between indexing benefits for read operations and the overhead introduced during write operations.
- Designing appropriate partitioning strategies that improve query performance without negatively impacting data retrieval for other query patterns.
- Understanding each ACID property and acknowledging its importance.
- Understanding different types of schedules (serializable,non serializable) and the concept of conflict serializability.
- The use of 2PL and its benefits in optimizing schedules and ensuring data integrity especially in a multi-user database or application.
- Simulating deadlock scenarios and understanding how the DBMS detects and resolves them through wait-for graph analysis.

5 Conclusion

In this lab,we explored several important physical design choices and transaction management concepts using our MNHS database.
We learned how indexes and partitioning can significantly impact query performance ,especially when working with very large databases .
This lab also helped us understand that this optimization comes with an overhead on insert and update operations and may not be beneficial for some queries.In addition,we revisited transaction management concepts like ACID properties,conflict serializability,and 2PL.
While working on different scenarios and schedules,we learned how concurrency control ensures data consistency ,integrity , and safe data access in a multi-user database.
Overall this lab highlighted the importance of carefully choosing physical design and concurrency strategies based on workload and data access patterns. It showed that good database performance and correctness are the result of informed design choices.